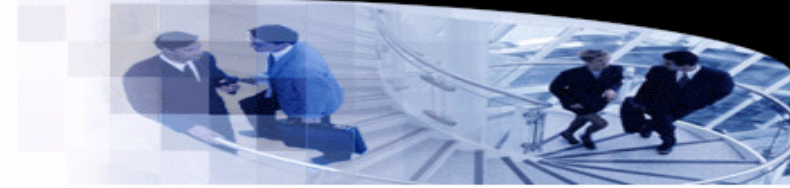




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

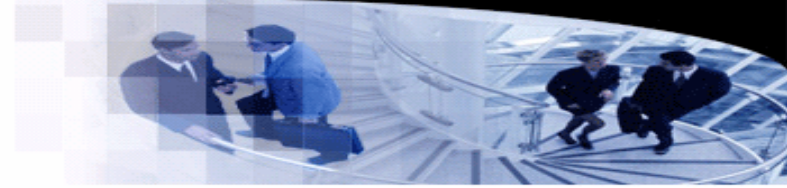
Lecture 2. Introduction of OCaml

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

About OCaml



- OCaml

- A general-purpose, strongly typed programming language.

- Important features

- First-class functions, static type checking and type inference, parametric polymorphism, algebraic data types, and pattern matching.

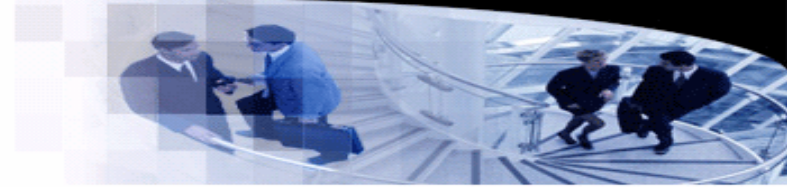
- Applications

- Jane Street Capital
- Bloomberg

- Some interesting projects

- Binary Analysis Platform: <https://github.com/BinaryAnalysisPlatform/bap>
- Google Drive Ocamlfuse: <https://github.com/astrada/google-drive-ocamlfuse>

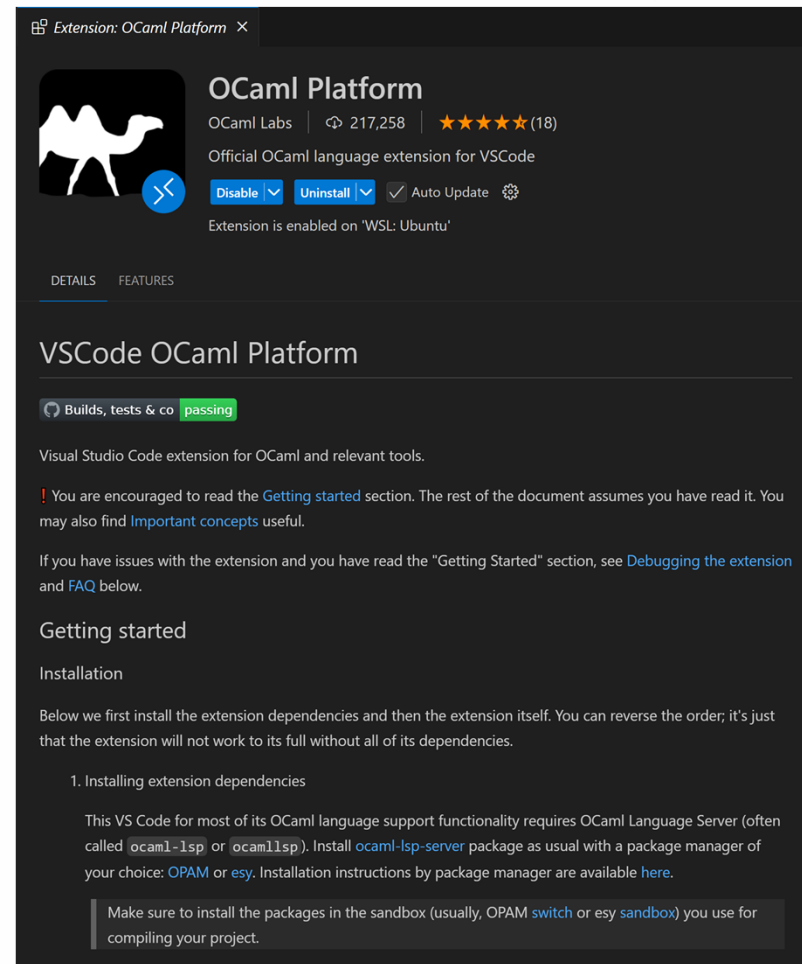
History of OCaml



- **ML**
 - The meta language of the LCF (Logic for Computable Functions) proof assistant, released by Robin Milner in 1972.
- **Caml**
 - Created by Ascánder Suárez as part of the Formel project at INRIA in 1987.
 - In 1990, Xavier Leroy and Damien Doligez built a new implementation called Caml Light that they based on a bytecode interpreter with a fast, sequential garbage collector.
- **OCaml**
 - Didier Rémy and Jérôme Vouillon implemented a powerful and elegant object system to form OCaml in 1996.

Getting Started with OCaml

- OCaml home
 - <https://ocaml.org/>
- Installation
 - <https://ocaml.org/install>
- Try OCaml online
 - <https://try.ocamlpro.com/>
- Exercises
 - <https://ocaml.org/exercises>
- IDE
 - VS Code with OCaml Platform



The screenshot shows the VS Code extension page for 'OCaml Platform' by OCaml Labs. The extension has 217,258 downloads and a 5-star rating from 18 reviews. It is the official OCaml language extension for VS Code. The page includes tabs for 'DETAILS' and 'FEATURES'. Under 'DETAILS', there is a status bar indicating 'Builds, tests & co passing'. The main content area provides instructions for getting started, including a warning to read the 'Getting started' section and a note about installing dependencies. A highlighted box at the bottom advises installing packages in the sandbox.

Extension: OCaml Platform ✕

OCaml Platform
OCaml Labs | 217,258 | ★★★★★ (18)
Official OCaml language extension for VSCode

Disable Uninstall ✓ Auto Update ⚙️
Extension is enabled on 'WSL: Ubuntu'

DETAILS FEATURES

VSCode OCaml Platform

Builds, tests & co **passing**

Visual Studio Code extension for OCaml and relevant tools.

! You are encouraged to read the [Getting started](#) section. The rest of the document assumes you have read it. You may also find [Important concepts](#) useful.

If you have issues with the extension and you have read the "Getting Started" section, see [Debugging the extension](#) and [FAQ](#) below.

Getting started

Installation

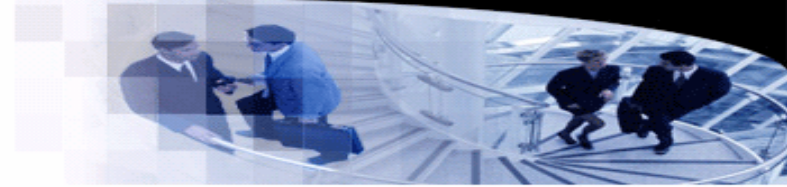
Below we first install the extension dependencies and then the extension itself. You can reverse the order; it's just that the extension will not work to its full without all of its dependencies.

1. Installing extension dependencies

This VS Code for most of its OCaml language support functionality requires OCaml Language Server (often called `ocaml-lsp` or `ocaml1lsp`). Install `ocaml-lsp-server` package as usual with a package manager of your choice: [OPAM](#) or [esy](#). Installation instructions by package manager are available [here](#).

Make sure to install the packages in the sandbox (usually, [OPAM switch](#) or [esy sandbox](#)) you use for compiling your project.

Hello World!



- hello.ml

```
ocaml > 🐪 hello.ml
1
2   let () = print_string "Hello world!\n"
3
4   |
```

- Compiling and executing

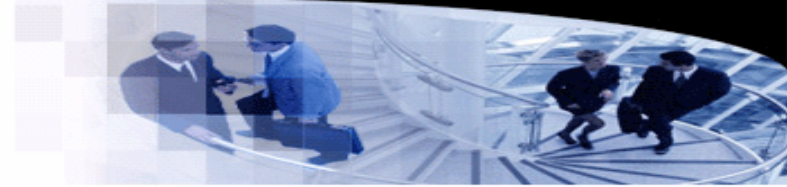
```
yml@DESKTOP-VPDMSDJ:~/ocaml$ ocamlc -o hello.out hello.ml
yml@DESKTOP-VPDMSDJ:~/ocaml$ ./hello.out
Hello world!
yml@DESKTOP-VPDMSDJ:~/ocaml$ █
```

Hello World!

- Toplevel system for Ocaml
 - A read-eval-print loop (REPL)

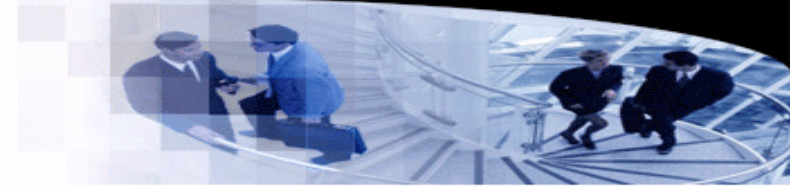
```
yml@DESKTOP-VPDMSDJ:~$ ocaml
OCaml version 5.2.1
Enter #help;; for help.

# let () = print_string "Hello world!\n";;
Hello world!
# 1 + 2;;
- : int = 3
#
```



Hello World!

- Toplevel system for Ocaml
 - A read-eval-print loop (REPL)



```
yml@DESKTOP-VPDMSDJ:~$ ocaml
OCaml version 5.2.1
Enter #help;; for help.

# let () = print_string "Hello world!\n" ;;
Hello world!
# 1 + 2;;
- : int = 3
#
```

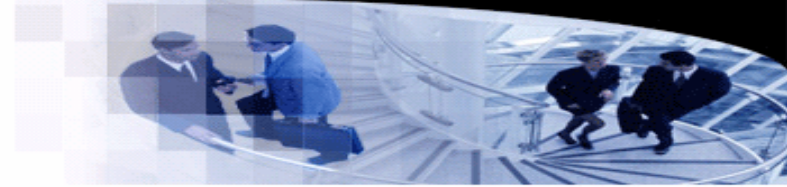
Invoke OCaml in the
interactive mode

Should be terminated by a
double-semicolon

Expression or definition

The result of the
evaluation

OCaml Program



- Program

- A sequence of definitions and expressions.

```
ymt@DESKTOP-VPDNSDJ:~/ocaml$ cat program.ml
```

```
let x = 2;;
```

```
let y = x + 2;;
```

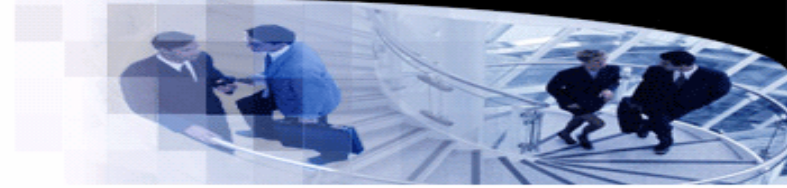
```
y + 3;;
```

```
print_int y;;
```

```
ymt@DESKTOP-VPDNSDJ:~/ocaml$ ocaml program.ml
```

```
4ymt@DESKTOP-VPDNSDJ:~/ocaml$ █
```


OCaml Program



- Program

- A sequence of definitions and expressions.

```
yml@DESKTOP-VPDNSDJ:~/ocaml$ cat program.ml
```

```
let x = 2;;
```

```
let y = x + 2;;
```

```
y + 3;;
```

```
print_int y;;
```

```
yml@DESKTOP-VPDNSDJ:~/ocaml$ ocaml program.ml
```

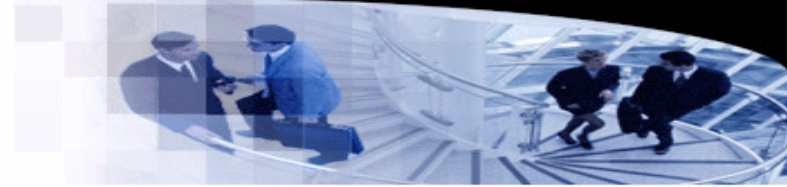
```
4
```

Definitions.
Definitions are also
expressions.

Expressions

Outputs

OCaml Program

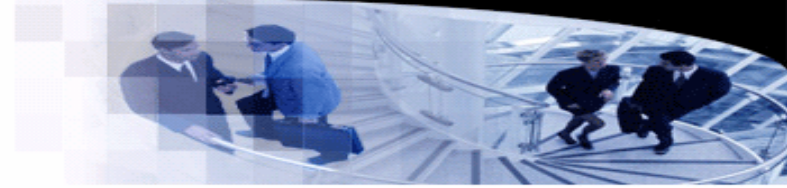


- Definition

- Bind a name with an expression.

```
# let x = 1;;  
val x : int = 1  
# x + 2;;  
- : int = 3  
#
```

OCaml Program



- Definition

- Bind a name with an expression.

Keyword

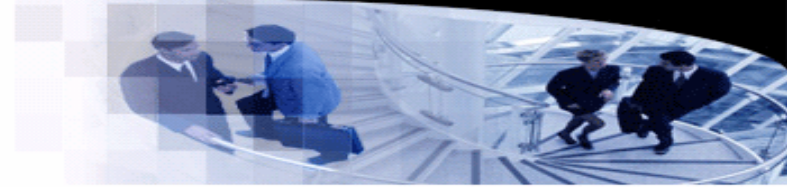
Name

Expression

Using the
bound
expression

```
# let x = 1;;  
val x : int = 1  
# x + 2;;  
- : int = 3  
#
```

OCaml Program

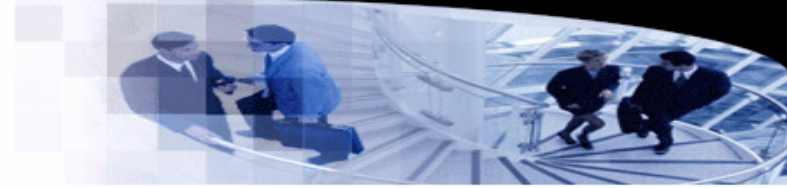


- Local bindings

- “`let x = e1 in e2`” binds name `x` with `e1` in `e2`.

```
# let x = 1;;  
val x : int = 1  
# x + 1;;  
- : int = 2  
# let y = x + 1 in y + 2;;  
- : int = 4  
# y  
  ii  
Error: Unbound value y  
# █
```

OCaml Program



- Local bindings

- “`let x = e1 in e2`” binds name `x` with `e1` in `e2`.

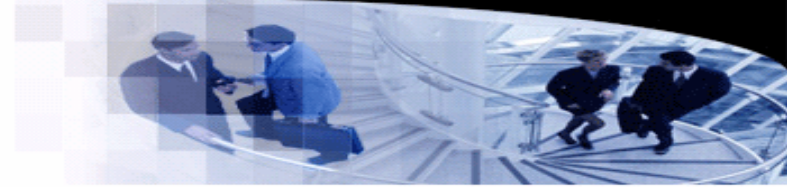
```
# let x = 1;;  
val x : int = 1  
# x + 1;;  
- : int = 2  
# let y = x + 1 in y + 2;;  
- : int = 4  
# y  
  ;;  
Error: Unbound value y  
#
```

x was bound globally.

y was bound in “y + 2”.

Out of scope.

OCaml Program

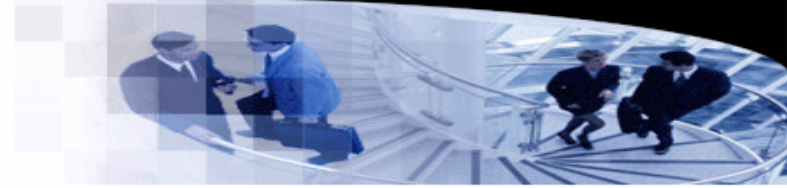


- Function definitions

- “`let f x = e`” defines a function `f`, which maps `x` to `e`.

```
# let f x = x * 2
;;
val f : int -> int = <fun>
#
```

OCaml Program



- Function definitions

- “let f x = e” defines a function f, which maps x to e.

Function name.

Parameter.

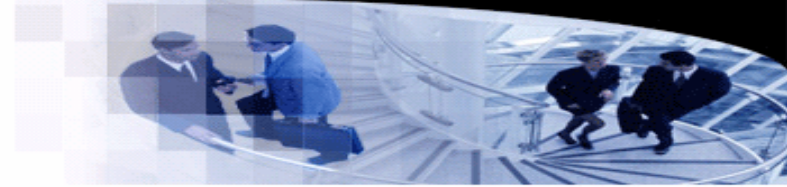
Mapped expression.

The value of this expression is a function.

```
# let f x = x * 2
;;
val f : int -> int = <fun>
#
```

The type of the function. In mathematics, $A \rightarrow B$ is the set of all possible functions from A to B .

OCaml Program



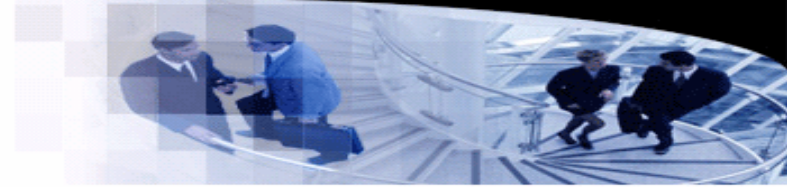
- Function definitions

- The types of functions are **inferred** by OCaml compilers.

```
# let f x = -x;;  
val f : int -> int = <fun>  
# let g x = -. x;;  
val g : float -> float = <fun>  
#
```

- In this example, “-.” is the float type version of minus.

OCaml Program

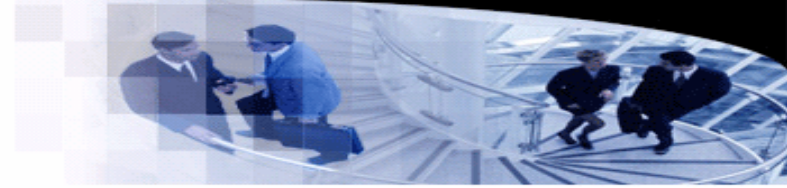


- Nameless functions (lambda terms)

- “`fun x -> e`” defines a function with no name that maps `x` to `e`.

```
# fun x -> x + 2;;  
- : int -> int = <fun>  
# (fun x -> x + 2) 3;;  
- : int = 5  
#
```

OCaml Program



- Functions with multiple parameters

- How to interpret the type of `plus`?

```
# let plus x y = x + y;;  
val plus : int -> int -> int = <fun>  
# plus 2 3;;  
- : int = 5  
#
```

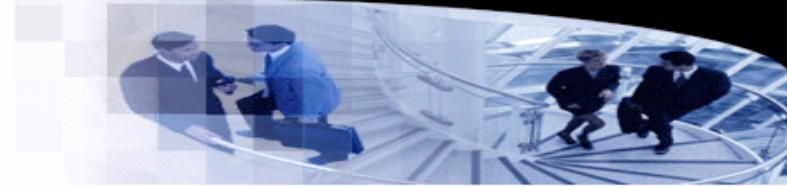
- “`->`” is right associate.
- `plus` is a function that maps an integer to another **function**, which maps an integer to another integer.

OCaml Program

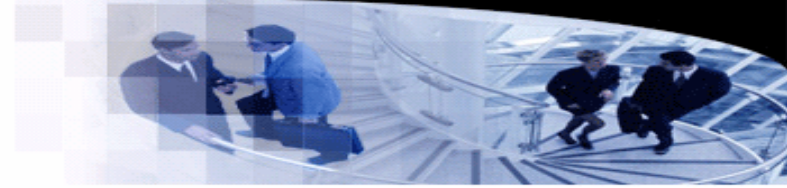
- Functions with multiple parameters

- How to evaluate “plus 2 3”?

```
# let plus x y = x + y;;  
val plus : int -> int -> int = <fun>  
# plus 2;;  
- : int -> int = <fun>  
# plus 2 3;;  
- : int = 5  
#
```



OCaml Program



- Functions with multiple parameters

- How to evaluate “plus 2 3”?

```
# let plus x y = x + y;;  
val plus : int -> int -> int = <fun>  
# plus 2;;  
- : int -> int = <fun>  
# plus 2 3;;  
- : int = 5  
#
```

2 is first applied to plus, which means x is replaced by 2.

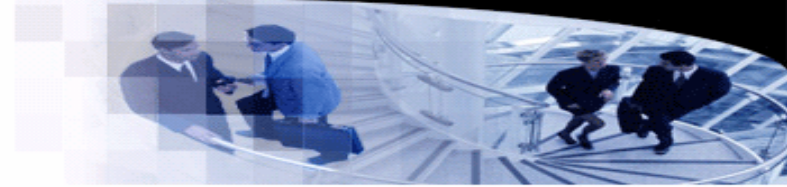
Since $(\text{plus } x \ y) = (\text{fun } y \rightarrow x + y)$, we have $(\text{plus } x \ y) \ 2 = (\text{fun } y \rightarrow 2 + y)$.

3 is then applied to (plus 2).

$\text{plus } 2 \ 3 = (\text{plus } 2) \ 3 = (\text{fun } y \rightarrow 2 + y) \ 3 = 2 + 3 = 5.$

- What about a function with no parameters?

OCaml Program



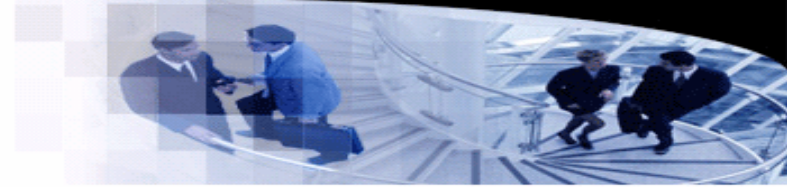
- Tuples

- `fst` and `snd` are predefined functions to access elements in pairs, which are tuples containing two values.

```
# (1, 2.0);;  
- : int * float = (1, 2.)  
# fst (1, 2.0);;  
- : int = 1  
# snd (1, 2.0);;  
- : float = 2.  
#
```

- To access elements in tuples containing more values, we will use pattern matching.

OCaml Program

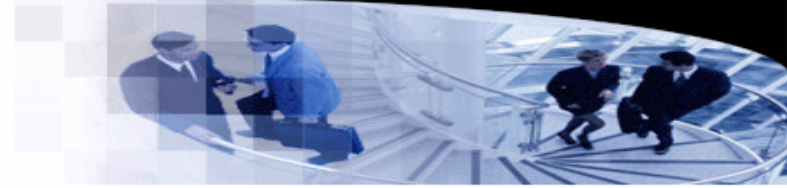


- Records

- A record is a bundle of values which can be accessed by their names.
- Record types must be declared before being used. Why?

```
# type person = {name: string; age: int};;  
type person = { name : string; age : int; }  
# let l = {name = "Lucy"; age = 20};;  
val l : person = {name = "Lucy"; age = 20}  
# l.name;;  
- : string = "Lucy"  
# l.age;;  
- : int = 20  
#
```

OCaml Program



- Variant types

- The declaration of a variant type lists all possible forms for values of that type.

```
# type integer = Int of int | ();;  
type integer = Int of int | ()  
# let x = Int 10;;  
val x : integer = Int 10  
# let y = ();;  
val y : integer = ()  
#
```

OCaml Program

- Variant types

- The declaration of a variant type lists all possible forms for values of that type.

```
# type integer = Int of int | ();;  
type integer = Int of int | ()  
# let x = Int 10;;  
val x : integer = Int 10  
# let v = ();;  
val v : integer = ()  
#
```

Constructor, a capitalized identifier.

The type integer is actually the set $int \cup unit$.

Hence, a value of integer can be 10 or ().

- Can we declare the type by just “`type integer = int | ();;`”?